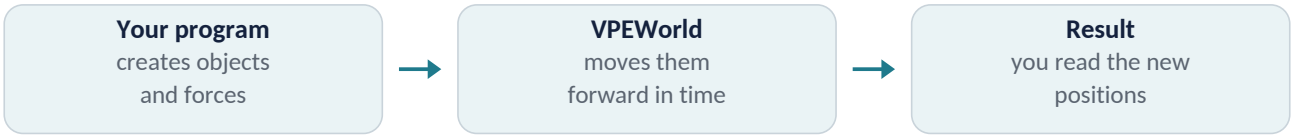
ViennaPhysicsEngine

Your first moving object in eight short pages.

ViennaPhysicsEngine (short: **VPE**) is a physics engine. It calculates how solid objects and pieces of cloth fall, slide and bump into each other. It only calculates. It does **not** open a window or draw anything. To see the objects, you send the results to a renderer, for example the Vienna Vulkan Engine (VVE).

In C++ you load the engine with `import VEPhysicsEngine;`. Everything in it is in the namespace `vpe`.



1. Install the tools

- **Git**, to download the code.
- **Visual Studio 2022 or newer**. In the Visual Studio Installer, select the workload *Desktop development with C++* and the component *C++ CMake tools for Windows*. This gives you the compiler, CMake and Ninja.
- For the 3D demo on page 8 you also need the **Vulkan SDK** and **vcpkg**. You can skip them for now.

2. Download the code

```
cd C:\data\GitHub
git clone https://github.com/hlavacs/ViennaPhysicsEngine.git
cd ViennaPhysicsEngine
```

This gives you the `main` branch, which is the current version. If you already have the folder, run `git pull` inside it instead of cloning again.

Word	Meaning in this guide
Module	A C++23 way to share code. You write `import` instead of `#include`.
CMake	A tool that reads `CMakeLists.txt` and prepares the build.
Ninja	The fast build program that CMake uses here.
Rigid body	A solid object that never bends, like a box or a ball.

What comes next: page 2 builds the engine, page 3 links it to your own program, page 4 explains the main classes, page 5 is a complete example, pages 6-7 add callbacks, joints and cloth, and page 8 shows the 3D demo and common problems.

In the code: README.md, section "Set up for Windows 11".

2. Build the physics engine

Start with the engine on its own. No graphics needed.

Open a normal PowerShell window in the `ViennaPhysicsEngine` folder and run:

```
.\build_windows.cmd debug --without-vve
```

The script finds Visual Studio by itself. `--without-vve` means "without the Vulkan engine": it builds only the physics library and its tests, and then runs the tests. Use `release` instead of debug for faster code. Add `--clean` to delete the old build folder first.

You see	It means
CMake messages	CMake found the compiler and prepared the build.
`[1/10] Building ...` lines	The library is being compiled.
`100% tests passed`	The engine works on your computer.
`Debug build complete.`	Done. The next line shows where the library file is.

The library ends up in `build\debug-windows-standalone\bin\debug\lib\ViennaPhysicsEngine.lib`. You do not need to copy it anywhere. Page 3 shows how your own program uses it.

The same build without the script

This is optional. Open the **x64 Developer PowerShell for VS** from the Start menu (it knows where the compiler is). The backtick ` at the end of a line continues the command on the next line.

```
cmake -S . -B build/standalone -G Ninja `
  -DCMAKE_BUILD_TYPE=Debug `
  -DVPE_BUILD_EXAMPLES=OFF -DVPE_BUILD_TESTS=ON
cmake --build build/standalone
ctest --test-dir build/standalone --output-on-failure
```

> **Two names for one thing**
>
> In `CMakeLists.txt` the library is called `ViennaPhysicsEngine::ViennaPhysicsEngine`. In your C++ code the module is called `VEPhysicsEngine`. Both mean the physics engine.

In the code: build_windows.cmd; CMakeLists.txt, line 158 (the library target).

3. Use the engine in your program

Put your project next to the engine folder.

Make a new folder `C:\data\GitHub\MyPhysics`. Put two files in it: `main.cpp` from page 5, and this `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.28)
project(MyPhysics LANGUAGES CXX)
set(VPE_BUILD_EXAMPLES OFF CACHE BOOL "" FORCE)
set(VPE_BUILD_TESTS OFF CACHE BOOL "" FORCE)
add_subdirectory(../ViennaPhysicsEngine
                 ${CMAKE_BINARY_DIR}/vpe)
add_executable(MyPhysics main.cpp)
target_link_libraries(MyPhysics PRIVATE
    ViennaPhysicsEngine::ViennaPhysicsEngine)
```

`../ViennaPhysicsEngine` only works if both folders are side by side in `C:\data\GitHub`.

Line	What it does
`set(VPE_BUILD_... OFF ...)`	Skips the engine's own demo and tests. You only want the library.
`add_subdirectory(...)`	Builds the physics engine together with your program.
`target_link_libraries(...)`	Connects your program to the engine. This also turns on C++23.

Then build and run it in the **x64 Developer PowerShell for VS**:

```
cd C:\data\GitHub\MyPhysics
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build build
.\build\MyPhysics.exe
```

What you should see

The program prints `Final height: 0.5005` and ends. No window opens, because this program has no renderer.

Common mistakes

Writing `import VEPhysicsEngine;` is not enough on its own: the `target_link_libraries` line must be there too. Do not `#include` the file `VPE.ixx`, and do not use the old `VPE.hpp` from version 1.

In the code: CMakeLists.txt (options at the top, library target at line 158).

4. The main parts of the engine

A world, the objects in it, and how they are connected.

Name	What it is
`vpe::VPEWorld`	The world. It holds all objects. `tick(seconds)` moves time forward.
`VPEWorld::Body`	One solid object: position, rotation, speed, mass and shape.
`VPEWorld::g_cube`	A ready-made cube shape. Pass `&VPEWorld::g_cube` when you make a body.
`addBody` / `eraseBody`	Put a body into the world, or take it out again.
`setForce` / `removeForce`	Push a body, for example with gravity. Each force has a number (ID).
`m_on_move` / `m_on_erase`	Your functions that the engine calls when a body moved or was removed.
`addConstraint`	Connect two bodies with a joint, for example a hinge.
`Cloth` / `addCloth`	A soft object made of triangles, like a flag.

Five facts to know

- **Inverse mass** means 1 divided by the mass. A body with mass 2 kg has inverse mass 0.5. Inverse mass **0** means "infinitely heavy": the body never moves, like a wall.
- The **owner** is a pointer to your own game object. The engine only uses it as a name tag, so every body needs its own owner, and the owner must stay alive while the body exists.
- There is always a flat **floor at height y = 0**. Up is the y direction.
- **Gravity is not automatic** for solid bodies. You add it yourself with `setForce` (page 5). Cloth gets gravity automatically.
- Bodies are kept in a `std::shared_ptr`, a smart pointer that deletes the object when nobody uses it anymore. Keep the world and the shape alive yourself: the body does not do that for you.

Small helpers such as `glmvec3` (a 3D vector), `glmquat` (a rotation) and `1.0_real` (a number in the engine's precision) can be used without `vpe::`. "No rotation" is written `glmquat{1,0,0,0}`.

> **Three rules that prevent most crashes**
> Give times in seconds. Use only convex shapes (no holes or dents), such as the built-in cube. Never remove a body while the engine is calling one of your callbacks: write it down in a list, and remove it after `tick` has finished.

In the code: modules/VPE.ixx, class VPEWorld (line 237), class Body (line 461), struct Force (line 268).

5. Example: a falling cube

The complete main.cpp for the project on page 3.

```
#include <iostream>
#include <memory>
import VEPhysicsEngine;

int main() {
    int owner = 1;                        // Our "game object".
    vpe::VPEWorld world;
    using W = vpe::VPEWorld;
    auto body = std::make_shared<W::Body>(
        &world, "cube", &owner, &W::g_cube,
        glmvec3{1,1,1}, glmvec3{0,4,0},
        glmquat{1,0,0,0}, glmvec3{0,0,0},
        glmvec3{0,0,0}, 1.0_real, 0.2_real, 0.8_real);
    body->setForce(0ul,                   // Force number 0:
        W::Force{{0, world.c_gravity, 0}});   // gravity.
    world.addBody(body);
    for (int frame = 0; frame < 120; ++frame)
        world.tick(1.0/60.0);             // 1/60 second.
    std::cout << "Final height: " << body->m_positionW.y
              << '\n';
    world.eraseBody(body);
}
```

Value in the code	Meaning
`&world, "cube", &owner, &W::g_cube`	The world, a name, the owner (name tag) and the shape.
`glmvec3{1,1,1}`, `glmvec3{0,4,0}`	Size 1 x 1 x 1, starting 4 units above the floor.
`glmquat{1,0,0,0}`	No rotation.
`glmvec3{0,0,0}` twice	Not moving and not spinning at the start.
`1.0_real`	Inverse mass 1, so the mass is 1.
`0.2_real`, `0.8_real`	Bounciness (0 = no bounce) and friction (how much it grips).

The loop calls `tick` 120 times with 1/60 second each, so two seconds pass. The cube falls and lands on the floor. The result is 0.5005: the center of a cube of height 1 that lies on the floor is at 0.5. The extra 0.0005 is a small safety margin the engine adds to every shape.

> **In a real game**
> Call `world.tick(secondsSinceLastFrame)` once per frame. To pause, simply do not call `tick`. When you continue, do not pass the whole paused time to `tick`, or the engine will try to catch up on all of it.

In the code: modules/VPE.ixx, Body constructor (line 535), tick (line 1234).

6. Callbacks and joints

Two small additions to the falling cube.

Get told when a body moves

A **callback** is a function you give to the engine, and the engine calls it later. Put this before `world.addBody(body);` on page 5. In a real game you would move the visible object here instead of printing.

```
body->m_on_move = [](double residual,
                     std::shared_ptr<W::Body> b) {
    std::cout << "y=" << b->m_positionW.y << '\n';
};
```

`residual` is the small bit of time left over since the last physics step. You can ignore it at first.

Connect two bodies

A **joint** holds two bodies together but still lets them move in some way. Here `bodyA` and `bodyB` are two bodies you have already added. The first vector is the meeting point in the world, the second is the hinge axis (it must not be zero).

```
auto hinge = std::make_shared<W::HingeJoint>(
    bodyA, bodyB, glmvec3{0,2,0}, glmvec3{0,1,0});
hinge->enableLimit(-0.5_real, 0.5_real); // Angles in radians.
world.addConstraint(hinge);
// Later, to remove the joint again:
world.removeConstraint(hinge);
```

Joint	Use it when the two bodies should...
`BallSocketJoint`	share one point but turn freely, like a shoulder.
`HingeJoint`	turn around one axis only, like a door.
`SliderJoint`	slide along one axis only, like a drawer.
`FixedJoint`	stay glued together.
`DistanceConstraint`	keep their centers at the same distance, like a rod.

The engine does not draw anything in a callback. Your game copies the new position to its visible object, and the renderer draws it. Do not delete objects inside a callback (see page 4).

In the code: modules/VPE.ixx, Body::m_on_move (line 479), HingeJoint (line 2082), addConstraint (line 1852).

7. Example: a hanging cloth

A small square of cloth, held at its two top corners.

Add `#include <vector>` at the top of `main.cpp`, and put this after `vpe::VPEWorld world;` and before the loop:

```
int clothOwner = 2;                    // Its own name tag.
std::vector<glmvec3> vertices{
    {-1,3,0}, {1,3,0}, {-1,1,0}, {1,1,0}};   // 4 corners.
std::vector<uint32_t> triangles{0,2,1, 1,2,3}; // 2 triangles.
std::vector<glmvec3> pinned{vertices[0], vertices[1]};
auto onMove = [](double, std::shared_ptr<W::Cloth> cloth) {
    auto positions = cloth->generateVertices();
    // Copy positions into your visible mesh here.
};
auto onErase = [](std::shared_ptr<W::Cloth>) {};
auto cloth = std::make_shared<W::Cloth>(
    &world, "sheet", &clothOwner, onMove, onErase,
    vertices, triangles, pinned, 0.1_real, 8, 0.8_real);
world.addCloth(cloth);
```

Input	Meaning
`vertices`	The corner points of the cloth.
`triangles`	Three numbers per triangle: which corners belong together.
`pinned`	Points that stay where they are. They must be the same as points in `vertices`.
`0.1_real`	How easily the cloth bends. Bigger means softer.
`8`	Small steps per physics step. More steps are more exact but slower.
`0.8_real`	How much of a movement you apply to the cloth is simulated (see chapter 18 of the manual).

`generateVertices()` returns the corners in the same order you gave them, so you can copy them straight into your visible mesh. Your renderer must then update the mesh on the graphics card.

> **Cleaning up**
> After the loop, call `world.clearCloths();`. Cloth is stored separately from solid bodies, so `world.clear()` does not remove it.

In the code: modules/VPE.ixx, class Cloth (line 3340), Cloth constructor (line 3456), addCloth (line 2882).

8. The 3D demo and common problems

See the physics on screen.

The demo program `physicsexample` draws the physics with the Vienna Vulkan Engine (VVE). You need:

- the **Vulkan SDK** (its installer sets `VULKAN_SDK`),
- **vcpkg**, either on the PATH or with `VCPKG_ROOT` set to its folder,
- **CMake 3.31.8 or newer** (Visual Studio's CMake is usually new enough),
- VVE downloaded **next to** VPE: `C:\data\GitHub\ViennaVulkanEngine` and `C:\data\GitHub\ViennaPhysicsEngine`.

```
cd C:\data\GitHub
git clone https://github.com/hlavacs/ViennaVulkanEngine.git
cd ViennaPhysicsEngine
.\build_windows.cmd debug
```

The script first downloads VVE's libraries with vcpkg, then builds VVE and VPE together. The first build takes a while. VVE is built as part of this build because C++ modules only fit together when both sides are compiled with exactly the same settings. Start the demo here: `ViennaVulkanEngine\bin\debug\exe\physicsexample.exe`.

Problem	Check this
The body does not move	Inverse mass above 0? Gravity added with `setForce`? Is `tick` called?
Nothing on screen	VPE alone draws nothing. Did you copy positions in a callback?
Crash after removing a body	Was the owner or shape deleted too early? Removed inside a callback?
Very slow after a pause	`tick` got a huge time and is catching up. Pass a small time instead.
Cloth constructor throws	Valid numbers? Three indices per triangle? Steps above 0?
`VULKAN_SDK is not set`	Install the Vulkan SDK, or build with `--without-vve`.

Updated 12 October 2026 for the main branch. The code on pages 5-7 was compiled and run again with Clang 18 (result: 0.5005). The full manual, vienna_physics_engine_v2_manual.pdf, explains how the engine works inside.

In the code: build_windows.cmd; examples/physicsexample/physicsexample.cpp.